Assignment 1 (10 Grades)

Single-Page Application (SPA) with PHP & MySQL

IS333 Web-Based Information Systems – Spring 2026

Team Size: 7-9 Students	Deadline: Friday 24-04-2026 @ 11:00 PM

Overview

In this assignment, you will design and build a **Single-Page Application (SPA)** of your own choice using PHP, MySQL, AJAX, and at least one third-party API. A Single Page Application (SPA) is a web application that loads a single HTML page and dynamically updates content via JavaScript as the user interacts with it, without reloading the entire page. This approach is commonly used in apps like Gmail, Google Maps, Airbnb, Netflix, Pinterest, and PayPal.

The project application topic selection is entirely up to your team – you are free to build anything meaningful: an email manger, a task manager, a recipe finder, a weather dashboard, a movie tracker, a fitness log, or any other idea that genuinely interests you.

The goal is to apply good web-development practices in a realistic context; you will be assessed on how well you implement the required technical features within your chosen application.

Requirements

1. Application Concept & UI

- **Choose a meaningful topic.** Your app should solve a real (even if small) problem or serve a clear purpose.
- **Single page feels.** All interactions happen on one page without full reloads – use AJAX / fetch (libraries like axios, jquery… etc are also allowed) for all data exchanges.
- **Header & Footer.** Include a styled header and footer as separate PHP include files (**header.php**, **footer.php**).
- **Responsive design.** The page must look reasonable on both desktop and mobile viewports.

2. Database Integration

- **MySQL database.** Design at least one table relevant to your application (e.g., tasks, recipes, bookmarks).
- **CRUD operations.** Implement at minimum Create and Read; Update and Delete.
- **No SQL injection.** Use prepared statements or PDO with parameterized queries – raw string concatenation is <u>not allowed</u>.

3. AJAX & Server-Side Logic

- **All data interactions use AJAX** (the Fetch API). The page must never do a full reload to show new data.
- **All CRUD operations are executed using AJAX calls:** for example, one to **insert** data and one to **retrieve / search for** it.
- **Server-side validation.** Validate all input in PHP before writing to the database. Return meaningful JSON error messages to the client.
- **Client-side validation.** Validate all required fields, formats, and constraints in JavaScript before sending the request.

4. Third-Party API Integration

Integrate **at least one public third-party API** that is relevant to your application. The API should enrich or extend what your app can do on its own. Some ideas:

Example APIs by application type
Weather app → OpenWeatherMap, WeatherAPI
Movie / book tracker → OMDb API, Open Library API, TMDB Recipes → Spoonacular, TheMealDB
Currency / Finance → ExchangeRate-API, CoinGecko
Translator → MyMemory, LibreTranslate User registration → WhatsApp Number Validator (RapidAPI – finestoreuk)
Or any other free/freemium API you find on RapidAPI, Postman Public APIs, or the public-APIs list.

- **The API call to be made from PHP** –. Place all API-related code in **API_Ops.php [**used to make the actual call to the API using CURL**]** and **API_Ops.js** [used to send ajax request to the server for API calls].
- **Handle API errors gracefully.** If the API is unavailable or returns an error, display a user-friendly message rather than crashing.
- **Do not hard-code API keys** in client-side JavaScript when using a secret key. Route the request through your PHP backend instead.

5. File / Media Upload

If your application naturally involves user-uploaded files (profile pictures, recipe photos, document attachments, etc.):

- **Store the file on the server** and save its name / path in the database.
- **Validate file type and size** on both client and server side.

Project Structure

Your project must contain the following PHP files. Each file has a single, well-defined responsibility:

File	Responsibility
`index.php`	Main application page. Includes header.php and footer.php. Contains the SPA HTML structure and inline or linked JavaScript.
`header.php`	Site header (logo, navigation, branding). Included at the top of index.php.
`footer.php`	Site footer (credits, links). Included at the bottom of index.php.
`DB_Ops.php`	Database connection, all INSERT / SELECT / UPDATE / DELETE functions, and server-side validation logic.
`Upload.php`	Handles file upload processing. Validates type, size, and moves the file.
`API_Ops.php` / `API_Ops.js`	All code for connecting to the chosen third-party API. Use PHP to hide secret keys; use JS to make the ajax call to your server.

Submission

- **Team members file.** Include a plain-text file **Team_Members.txt** containing team number, and each member's full name and student ID.
- **Database backup.** Export your database as a **.sql** file and include it in the submission folder.
- **Compressed archive.** Place all files in a single zip / rar folder named: YourTeamNumber**_ASSIGNMENT-1** (e.g., **15_ASSIGNMENT-1.zip**).
- **Upload to the Classroom submission link** posted by the instructor before the deadline.

⚠ Submissions that do not follow the naming convention or are missing the database backup will receive a 1-mark deduction.

Good Practices Checklist

Before submitting, verify that your project meets the following standards:

Code Quality
☐ Prepared statements used for all database queries (no raw string concatenation, you can use bind_param function)
☐ PHP errors suppressed in production; use error reporting only during development
☐ JavaScript separated from HTML (in a <script> block or external .js file)
☐ No inline styles; use a CSS file.
Security & Robustness
☐ API keys stored server-side (never exposed in client-side JS for secret keys)
☐ Server-side validation covers all user inputs before any DB write
☐ AJAX responses checked for errors; user shown a clear message on failure
☐ HTML output escaped with htmlspecialchars () to prevent XSS
Submission
☐ Team_Members.txt present with team number and all member names/IDs
☐ Database .sql backup included
☐ Folder named correctly: YourTeamNumber_ASSIGNMENT-1
☐ All files open without errors on a fresh XAMPP / WAMP / LAMP setup